

JWT Attacks

JWTs (JSON Web Tokens) are the natural next step after your Sessions and Cookies guides — where a traditional session ID is just a random lookup key referencing server-side state, a JWT is a **self-contained** credential that carries its own data and its own proof of authenticity. That self-containment is exactly what makes JWTs powerful for distributed systems, and exactly what makes flawed JWT handling so catastrophic when it goes wrong.

1. What a JWT Actually Is

Unlike a classic session token — which is just an opaque, random string of moderate size in cookies referencing server-side session data (your Sessions guide) — a JWT contains **all the data the server needs directly within the token itself**, cryptographically signed so the server can trust it wasn't tampered with. This makes JWTs a natural fit for highly distributed applications where multiple back-end servers need to independently verify a user's identity without a shared central session store.

Structure — three base64url-encoded parts, separated by dots:

```
eyJraWQiOiI5I... . eyJpc3MiOiJwb3J0c3dpZ2dldciIs... . SYZBPIBg2CRj...  
  HEADER                PAYLOAD                SIGNATURE
```

Decoded header:

```
{ "kid": "9136ddb3-...", "alg": "RS256" }
```

Decoded payload (the "claims"):

```
{  
  "iss": "portswigger",  
  "exp": 1648037164,  
  "name": "Carlos Montoya",  
  "sub": "carlos",  
  "role": "blog_author",  
  "email": "carlos@carlos-montoya.net",  
}
```

```
"iat": 1516239022  
}
```

Critical fact to internalize first: the header and payload are just **base64url-encoded JSON** — not encrypted, not obfuscated. Anyone holding the token can decode and read every claim trivially. The **only** thing standing between an attacker and freely modifying those claims is the signature — this single sentence is the entire premise of every attack in this guide.

2. JWT vs. JWS vs. JWE — A Naming Clarification

```
JWT    → the general spec, defines the CLAIMS format only  
JWS    → JSON Web Signature – the token's contents are signed but readable (what people mean 99% of the time they say "JWT")  
JWE    → JSON Web Encryption – the token's contents are actually encrypted, not just signed
```

When anyone (including this guide, and PortSwigger's own materials) says "JWT," they almost always mean a **JWS** token — signed but plainly readable. Worth knowing JWE exists as the encrypted variant, but the vulnerabilities in this guide are overwhelmingly about JWS-style signed-but-readable tokens.

3. Why JWT Vulnerabilities Are So Severe

JWTs are most commonly used precisely where the stakes are highest: **authentication, session management, and access control**. If an attacker can create their own valid, arbitrarily-crafted token, they can escalate their own privileges or impersonate any other user outright — this is functionally identical in impact to the session fixation/hijacking outcomes from your Sessions guide, just reached through a completely different mechanism (forging trust in a token's *content*, rather than stealing or fixing a session *identifier*).

4. The Root Cause of Nearly Every JWT Vulnerability

Because the server stores nothing about issued tokens (that's the whole point of JWTs being self-contained), **the server has no independent memory of what a token's original contents or signature should be**. If signature verification is flawed in any way, there is *nothing else* stopping an attacker from freely modifying claims:

```
{ "username": "carlos", "isAdmin": false }
```

Flip `isAdmin` to `true`, and if the server's signature check is broken, you've just handed yourself admin privileges — no exploitation technique beyond "edit a JSON field" required, *provided* the signature check itself can be defeated. Everything in this guide is really just different ways of defeating that one check.

5. Attack 1 — Accepting Arbitrary Signatures (No Verification At All)

Most JWT libraries provide two distinct methods: one that **verifies** the signature (`verify()`), and one that just **decodes** the token without checking anything (`decode()`). A genuinely common developer mistake is calling the decode-only method and never actually verifying the signature at all.

If the application only ever calls `decode()` → ANY signature is accepted, even a completely garbage one → an attacker can modify any claim and resign with literally anything, since nothing is ever checked

Testing methodology: take a valid token, modify a claim (e.g., `role: "admin"`), and replace the signature with anything — random garbage, an empty string, whatever — and resubmit. If the server still accepts it, verification isn't happening at all.

6. Attack 2 — The `alg: none` Attack

The JWT header's `alg` parameter tells the server which algorithm to use when verifying the signature — and this parameter is **attacker-controlled**, since it's part of the token the attacker is submitting. This is inherently dangerous: the server is being told, by untrusted input, how to check whether that same untrusted input can be trusted.

The JWS spec defines a special value: `alg: "none"`, representing an intentionally **unsigned** token:

```
{ "alg": "none", "typ": "JWT" }
```

```
header.payload.      ← note: the trailing dot after payload is still
                      required even though the signature is empty
```

Servers generally reject `none` explicitly — but since this filtering usually relies on simple string matching, it can sometimes be bypassed with classic obfuscation:

```
"none" → "None" → "NONE" → "nOnE" (mixed capitalization)
      → unexpected encoding variants of the string
```

This is genuinely the same lesson as your SQLi/command-injection guides' repeated warning about blacklist filtering being fragile — a naive equality/substring check against the literal string `"none"` is exactly the kind of check that case variation defeats.

7. Attack 3 — Brute-Forcing the Secret Key

Algorithms like **HS256** (HMAC + SHA-256) use a single shared secret string as the signing key — functionally similar to a password. If that secret is weak, default, a placeholder left over from copy-pasted example code, or otherwise guessable, an attacker can brute-force it entirely offline.

```
hashcat -a 0 -m 16500 <jwt> <wordlist>
```

This tests a wordlist of known/common JWT secrets against the token, signing the header+payload with each candidate secret and comparing the result to the token's actual signature. A well-known wordlist of leaked/default JWT secrets exists specifically for this (search "jwt-secrets wallarm" — a widely-referenced list).

Why hashcat specifically: the entire brute-force runs **locally**, with zero requests sent to the target server — meaning it's extremely fast even against huge wordlists, and generates no server-side logs or rate-limiting triggers at all, unlike an online password brute-force.

Once the secret is found: you can sign **any** header/payload combination yourself, since you now possess the exact same secret the legitimate server uses — full forgery capability, not just a one-off bypass.

8. Attack 4 — JWT Header Parameter Injection

Beyond `alg`, several other header (JOSE header) parameters tell the server **which key** to use for verification — and since these are also attacker-controlled, manipulating them can trick the server into using a key the attacker chose, instead of its own legitimate secret.

a) `jwk` — Embedded Public Key

The `jwk` parameter lets a token embed its own public key directly in JWK (JSON Web Key) format:

```
{
  "kid": "ed2Nf8sb-...",
  "alg": "RS256",
  "jwk": { "kty": "RSA", "e": "AQAB", "kid": "ed2Nf8sb
-...", "n": "yy1wpYmff..." }
}
```

Properly configured servers should only trust an **allowlist** of known public keys — but misconfigured servers sometimes trust **whatever key is embedded in the token itself**. If so:

1. Generate your own RSA key pair
2. Sign a modified token (with your desired forged claims) using YOUR private key
3. Embed YOUR matching public key in the `jwk` header
4. Server verifies the signature against the embedded key — which IS your key, so verification "succeeds," even though the real server secret was never involved at all

Burp's **JWT Editor extension** automates this entire attack via an "Embedded JWK" option once you've generated a key in its JWT Editor Keys tab — genuinely worth installing if you're practicing this in PortSwigger's labs.

b) `jku` — JWK Set URL

Instead of embedding the key directly, `jku` tells the server to **fetch** the verification key from a URL:

```
{ "jku": "https://attacker.com/jwks.json" }
```

If the server doesn't strictly allowlist which hosts it's willing to fetch keys from, an attacker hosts their own JWK Set at a URL they control, points `jku` at it, and signs the forged token with their own matching private key. **Directly connects to your XXE guide's SSRF discussion and your Open Redirect guide's Section 6b** — the same URL-allowlist-bypass techniques (subdomain tricks, userinfo tricks) covered in those guides apply identically here if the server's `jku` host validation is naive.

c) `kid` — Key ID

`kid` tells the server which specific key (often from a JWK Set, or a database/file lookup) to use when multiple keys are in play. The JWS spec doesn't define any required structure for this string — it's entirely developer-chosen, which means it's frequently implemented as a raw filename or database lookup key, with all the injection risk that implies:

Path traversal via `kid`:

```
{ "kid": "../../../path/to/file", "alg": "HS256" }
```

If the server reads the file at the `kid` path and uses its **contents** as the HMAC signing secret, an attacker who can point `kid` at a **predictable file** can then sign their forged token using that file's known contents as the secret:

```
kid: "../../../../dev/null"
```

`/dev/null` is present on virtually every Linux system and is always empty — so signing the token with an **empty string** as the secret produces a valid signature the server will accept, since it reads that same empty file as its verification key. A genuinely elegant, minimal-knowledge attack.

SQL injection via `kid`: if verification keys are looked up from a database using the `kid` value directly in a query, this is a **fresh SQL injection surface** — worth testing with everything from your SQLi guide, just via this specific header parameter as the entry point instead of a normal form field.

d) Other Notable Header Parameters

```
cty (Content Type) → if signature bypass is already achieved, injecting
                    cty: "text/xml" or cty: "application/x-java-serialized-object"
t"
```

can open up XXE (your XXE guide) or deserialization attack surface within the JWT processing pipeline itself
x5c (X.509 cert chain) → similar self-signed-certificate injection concept to jwk, with its own history of parser-specific CVEs due to the complexity of the X.509 format

9. Attack 5 — Algorithm Confusion (Deep-Dive Topic)

Even when secrets are strong and header injection is patched, a more advanced technique remains: **algorithm confusion**.

Symmetric algorithms (HS256) → ONE shared secret, used for both signing AND verifying
Asymmetric algorithms (RS256) → a PRIVATE key signs, a DIFFERENT PUBLIC key verifies – the public key is, by design, not secret at all

The core vulnerability: if a server supports both algorithm types but doesn't strictly enforce which one a given token *should* use, an attacker can:

1. Obtain the server's PUBLIC key (often exposed at a standard endpoint, or derivable from existing signed tokens)
2. Craft a forged token with modified claims
3. Change alg from RS256 to HS256 in the header
4. Sign the token using the server's PUBLIC key AS IF it were an HS256 shared secret
5. The server, expecting HS256, uses what IT believes is "the shared secret" to verify – but that's actually its own PUBLIC key, which the attacker just used correctly, since public keys are never actually secret

This is genuinely one of the more conceptually elegant JWT attacks — it exploits the server trusting the *attacker-supplied* `alg` header to decide which verification strategy to apply, exactly the same root architectural flaw as the `alg: none` attack from Section 6, just applied to a subtler confusion between two legitimate algorithm families instead of disabling verification outright.

10. Working With JWTs Practically (Burp/ZAP)

1. Proxy all traffic through Burp/ZAP (per your OWASP ZAP guide)
2. Install the JWT Editor extension (Burp) for structured decode/edit/re-sign functionality rather than manual base64 encoding/decoding
3. Capture a request containing a JWT, send to Repeater
4. Decode, modify claims, and test each attack from this guide systematically:
 - Try decode-only bypass (garbage signature)
 - Try alg: none with capitalization variants
 - Brute-force the secret with hashcat if HS256
 - Try jwk/jku/kid header injection
 - Try algorithm confusion if both symmetric/asymmetric are supported

`jwt.io`'s debugger is also genuinely useful for manually experimenting with token construction while you're still building intuition for the format, before moving to Burp's more testing-oriented tooling.

11. Practical Lab Example

PortSwigger's Web Security Academy has a full, dedicated JWT lab set covering every attack in this guide individually (signature bypass, alg:none, secret brute-forcing, jwk/jku/kid injection, algorithm confusion) – genuinely the most directly applicable hands-on environment, since this entire guide is built from their materials.

Suggested order (mirrors the escalating sophistication of this guide):

1. Unverified signature lab (Section 5)
2. alg: none lab (Section 6)
3. Weak secret / hashcat brute-force lab (Section 7)
4. jwk header injection lab (Section 8a)
5. kid path traversal lab (Section 8c)
6. Algorithm confusion lab (Section 9)

12. Prevention — Securing JWT Implementations

- Use a current, well-maintained JWT library, and ensure developers understand its `verify()` vs `decode()` distinction explicitly
- Perform ROBUST signature verification on every token, every time - never call a decode-only method and treat that as sufficient
- Explicitly reject alg: none, and account for case/encoding variations when filtering it
- Enforce a strict, server-side ALLOWLIST for the jku header - never trust an arbitrary attacker-supplied URL
- Validate the kid parameter strictly - never use it directly in a filesystem path or SQL query without validation (the same

- allowlist-over-blocklist principle from every injection guide in this series)
- Never allow the token's own claimed algorithm to silently switch the server's verification strategy between symmetric and asymmetric modes - the expected algorithm should be a fixed server-side configuration, not something read from the attacker-supplied token

Additional best practices (defense-in-depth, beyond strictly avoiding vulnerabilities):

- Always set an expiration (exp) claim
- Avoid sending JWTs in URL parameters (same leakage risks covered in your Sessions guide, Section 3)
- Include an aud (audience) claim specifying the intended recipient, preventing a token issued for one service being replayed against a completely different one
- Build in a genuine revocation mechanism - since JWTs are self-contained and stateless by design, a compromised or logged-out token otherwise remains valid until it naturally expires, which is a structural trade-off worth being explicit about compared to server-side sessions (your Sessions guide, Section 8), which can be invalidated immediately on demand

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-347 (Improper Verification of Cryptographic Signature) — the parent weakness behind nearly every attack in this guide; CWE-798 (Hard-coded Credentials) when a default/weak secret is involved
OWASP Top 10:2025	A07:2025 Authentication Failures (JWTs are explicitly an authentication/session mechanism); A08:2025 Software or Data Integrity Failures also applies, since signature verification IS integrity verification
CVSS	Routinely Critical — successful forgery typically grants full authentication bypass and often privilege escalation in a single step, AV:N/PR:N/UI:N/C:H/I:H common
Cyber Kill Chain	Exploitation — a forged token is a direct authentication-bypass exploit, structurally identical in outcome to the session hijacking/fixation techniques from your Sessions guide
MITRE ATT&CK	T1550 (Use Alternate Authentication Material), specifically overlapping with T1606 (Forge Web Credentials)

Quick Reference Cheat Sheet

Structure: header.payload.signature (all base64url, NOT encrypted)

Attack checklist:

1. Decode-only bypass → resign with garbage, see if it's still accepted
2. alg: none → try case variations (None, NONE, nOnE)
3. Weak secret → hashcat -a 0 -m 16500 <jwt> <wordlist>
4. jwk injection → embed your own key + self-signed token
5. jku injection → host your own JWK Set, bypass host allowlist
6. kid path traversal → point at a predictable file (/dev/null)
7. kid SQL injection → if keys are DB-looked-up via kid
8. Algorithm confusion → sign with server's PUBLIC key as if it were an HS256 shared secret

Prevention:

Always verify() not decode() | Reject alg:none robustly | Allowlist jku hosts | Validate kid strictly | Fix expected algorithm server-side, never trust the token's own alg claim to choose verification mode